

ReSTIR、UE 5.8 与 DSMEngine：源码级分析与落地路线

原始 ReSTIR、Lumen ReSTIR Gather、MegaLights 与 DSMEngine
落地路线

技术研究交付 · 2026-08-16

所有算法结论均以随附论文和指定源码基线为准

目录

执行摘要	5
1. 三种系统的正确关系	5
2. 原始 ReSTIR DI 的算法契约	6
2.1 它估计什么	6
2.2 为什么 DI 不使用 Lumen 的 Jacobian	6
2.3 偏置模式不是 “写错公式”	6
3. UE 5.8 Lumen ReSTIR Gather 源码解析	6
3.1 入口、开关与适用条件	6
3.2 样本与 reservoir 数据	7
3.3 Initial Samples	7
3.4 Temporal Resampling	7
3.5 Spatial Resampling	8
3.6 Integrate、上采样与材质响应	8
3.7 第二段时域滤波与双边滤波	8
3.8 Lumen 路径对 DSMEngine 的可借鉴点	9
4. UE 5.8 MegaLights 源码解析	9
4.1 产品目标与顶层流水线	9
4.2 候选来源：clustered light grid	9
4.3 向量化 WRS	10
4.4 样本与光线压缩	10
4.5 可见性后端	10
4.6 Resolve 与着色	10
4.7 历史到底保存什么	11
4.8 去噪	11
5. 原论文与 UE 的逐项差异	11
5.1 与 Lumen ReSTIR Gather	11
5.2 与 MegaLights	11
6. DSMEngine 当前渲染架构审阅	12
6.1 现有 frame pipeline	12

6.2 GBuffer 能提供什么 12

6.3 灯光系统 12

6.4 运动向量与历史 13

6.5 DXR 基础设施 13

6.6 TAA 与 ReSTIR 去噪的相互作用 13

7. 建议先冻结的外部契约 13

7.1 第一版功能范围 13

7.2 质量契约 14

7.3 性能契约 14

8. 推荐的 DSMEngine 架构 14

8.1 模块分层 14

8.2 Reservoir 参考布局 15

8.3 稳定灯光表 15

9. 建议算法：从 reference 到 UE 风格 15

9.1 Pass 0: brute-force reference 15

9.2 Pass 1: Initial RIS 16

9.3 枚举式 WRS 的正确接法 16

9.4 Pass 2: Temporal Resampling 16

9.5 Pass 3: Spatial Resampling 17

9.6 Pass 4: Visibility 与 Shade 17

9.7 Pass 5: 重建与去噪 17

9.8 Pass 6: 吸收 MegaLights 优化 18

10. 分阶段实施路线（分析，不执行） 18

阶段 A: 契约与诊断基础 18

阶段 B: Renderer 级 ray tracing scene 18

阶段 C: 单帧正确性 18

阶段 D: 时间复用 19

阶段 E: 空间复用 19

阶段 F: 重建与 UE 风格优化 19

阶段 G: 扩展采样域 20

11. 验证矩阵 20

11.1 正确性场景 20

11.2 数值指标 20

11.3 性能指标 20

12. 主要风险与应对21

风险 1：在 128 灯上没有性能收益 21

风险 2：TLAS 成本吞掉采样收益 21

风险 3：历史灯光索引失配 21

风险 4：重复累计和不可解释偏差 21

风险 5：高光重尾与低分辨率漏样 21

风险 6：误用 GI Jacobian 21

风险 7：RayQuery 平台路径未验证 21

13. 参数起点与调优顺序21

14. 是否应“完全仿照 UE”22

应仿照 22

不应仿照 22

15. 决策门23

16. 最终建议23

17. 取证来源与版本说明23

论文与官方资料 23

UE 5.8.1 本机源码 24

DSMEngine 源码范围 24

分析对象：Bitterli 等 2020 年 ReSTIR DI 原论文、UE 5.8.1 源码中的 Lumen ReSTIR Gather 与 MegaLights、DSMEngine 当前延迟渲染与 DXR 基础设施

源码基线：本机 G:\Works\QSCClient，Build.version 为 UE 5.8.1，兼容 CL 55116800；该源码树包含 QQSpeed 定制标记，因此本文会区分 Epic 架构与本地分支细节

DSMEngine 基线：D:\Code\DSMEngine，2026-08-16 只读审阅

交付边界：只做分析与设计，不修改引擎实现

执行摘要

UE 中并不存在一个可以简单称为“UE 的 ReSTIR DI”的单一模块。源码里最接近论文名义和数据结构的是 Lumen ReSTIR Gather，但它采样的是一次间接反弹方向，属于实验性的 ReSTIR GI；面向大量直接光、也最接近用户期望效果的是 MegaLights，它采用并行加权随机抽样、历史可见性引导和时空去噪，却没有把历史 reservoir 作为下一帧 proposal 合并，因此不是经典的时空 ReSTIR DI。

对 DSMEngine，最合理的目标不是复制 Lumen ReSTIR Gather，而是实现一个边界清楚的 ReSTIR DI：第一阶段只支持不透明表面的点光和聚光灯，方向光继续走现有路径，最终每像素对选中局部灯追踪一条阴影光线；在正确性参考实现稳定后，再吸收 MegaLights 的向量化灯光枚举、可见/隐藏哈希引导、压缩样本与去噪经验。

当前项目具备 GBuffer、运动向量、延迟光照和底层 DXR 抽象，但缺少四个关键契约：稳定灯光身份、Renderer 级 TLAS 生命周期、ReSTIR 专用历史有效性、可验证的 RayQuery/阴影光线入口。先补齐这些基础，再写 reservoir shader，风险远低于直接插入一个“ReSTIR pass”。

还有一个必须客观看待的性能结论：DSMEngine 目前局部灯上限只有 128，现有 tiled raster lighting 对无阴影局部灯可能更便宜。ReSTIR 的近期价值主要是把大量局部灯的光线追踪阴影成本压到近似固定预算，以及为未来面光/发光几何扩展奠定采样框架；是否更快必须由场景与 GPU 计时证明。

1. 三种系统的正确关系

维度	2020 ReSTIR 原论文	UE Lumen ReSTIR Gather	UE MegaLights
目标	动态多光源直接光	Lumen 一次间接漫反射/粗糙高光 gather	大量随机直接光
被选样本	光源/发光三角形上的点	首次命中点发出的半球射线及其二次命中	解析灯光及面内参数
初始候选	功率采样灯光/环境	每像素均匀半球方向	clustered light grid 中枚举/加权抽样
历史 reservoir	有	有	无
空间 reservoir 合并	有	有	无；后端只滤光照
几何 Jacobian	DI 光源域不需要 GI Jacobian	需要 ReSTIR GI Jacobian	不适用
可见性	选后阴影测试并可传播	对 GI sample 做重追踪/屏幕遮挡测试	屏幕追踪、VSM 或硬件/软件 RT
后端重建	论文主要讨论估计器	上采样、双边与第二段时域滤波	专用时空去噪、置信度与哈希历史
成熟度	研究算法	源码标记为实验/原型	UE 产品化直接光功能

因此“原论文与 UE ReSTIR 的区别”要拆成两问：

1. 原论文与 Lumen ReSTIR Gather 的区别，核心是 DI 对 GI、光源样本对路径样本、无 Jacobian 对有 Jacobian；

2. 原论文与 MegaLights 的区别，核心是 时空 reservoir 重采样对每帧 WRS + 历史引导 + 光照去噪。

2. 原始 ReSTIR DI 的算法契约

2.1 它估计什么

原论文只估计相机首个表面点的直接光照：

$$f(x,y) = Le(y \rightarrow x) * BSDF(x,y) * G(x,y) * V(x,y)$$

候选从可采样分布 $p(y)$ 产生，以未归一化目标 $pHat_x(y)$ 重新加权：

$$w(y) = pHat_x(y) / p(y)$$

reservoir 保存选中样本 y 、累计权重 $wSum$ 、代表候选数 M ，最终：

$$W = wSum / (M * pHat_x(y))$$

$$L_hat = f(x,y) * W$$

合并来自另一个像素/帧的 reservoir 时，等效权重为：

$$wMerge = pHat_current(R.y) * R.W * R.M$$

这四条式子定义了外部契约。任何 UE 风格优化都必须说明它是在保持、近似还是主动打破哪一条。

2.2 为什么 DI 不使用 Lumen 的 Jacobian

原论文的 y 位于灯光采样域。接收点从邻居移动到当前像素时，算法在当前点重算 $pHat_current(y)$ ，而光源面积域样本本身没有改换成另一个路径顶点测度。

ReSTIR GI 不同：复用的是邻居路径的第二顶点。把同一世界空间命中点连接到新的首顶点后，局部半球立体角与表面面积之间的映射变了，需要：

$$J(q \rightarrow r) = |\cos(\phi_i_r)| / |\cos(\phi_i_q)|$$

$$* |x1_q - x2_q|^2 / |x1_r - x2_q|^2$$

把 Lumen shader 中的这个 Jacobian 照搬到解析灯光 DI，会把两个不同的测度变换混为一谈。

2.3 偏置模式不是“写错公式”

朴素空间/时间合并忽略不同 proposal 的完整 MIS 支撑，会产生偏差。原论文同时提供昂贵的无偏修正和更实用的偏置变体。工程实现可以选择后者，但应做到：偏差来源显式、clamp 参数可控、有 brute-force reference、有静态场景平均误差验证。不能用“时域稳定”掩盖系统性丢能量。

3. UE 5.8 Lumen ReSTIR Gather 源码解析

3.1 入口、开关与适用条件

主要文件为：

- Engine/Source/Runtime/Renderer/Private/Lumen/LumenReSTIRGather.cpp/h;
- Engine/Shaders/Private/Lumen/LumenReSTIRGather.usf;

- `LumenScreenProbeGather.cpp` 中的 final gather 分派；
- `LumenViewState.h` 中的历史资源。

该路由 Lumen final gather 方法选择进入，只在 SM6、硬件光追与相应 Lumen 条件满足时启用。源码和 CVar 命名都把它定位为实验性路径，而不是通用的独立 ReSTIR 插件。默认 reservoir 分辨率相对输出下采样 2 倍，即一个 reservoir 覆盖 2x2 输出像素附近的信息。

它依赖 Lumen 已有能力：surface cache、硬件光追 hit lighting、天空光处理、pre-exposure、短程 AO、Lumen 历史与 scene update。脱离 Lumen 单独移植 shader 并不能工作。

3.2 样本与 reservoir 数据

HLSL 中的 `FSample` 近似包含：

```
RayDirection + PDF
OutgoingRadiance
HitDistance
HitNormal
```

`FReservoir` 包含样本以及 `wSum`、`M`、`W`。资源层面分别保存方向/PDF、radiance、距离、法线和权重；颜色常用 `R11G11B10`，距离为 `R32F`，法线为 10:10:10:2，权重使用浮点 RGBA。拆分纹理便于各 pass 只读所需数据，也便于历史提取和 ping-pong。

被复用的不是“某一盏灯”，而是“从当前首命中点发出的一条半球射线，以及这条射线命中的二次表面所返回的 outgoing radiance”。这决定了后续为什么要保存 hit normal/distance，为什么要做几何 Jacobian，也决定了它无法直接解决大量解析灯的直接阴影。

3.3 Initial Samples

每个 reservoir 像素先用 blue-noise 随机数生成均匀半球方向，PDF 随方向分布确定。硬件光追沿这个方向查询场景：

- 命中后，可通过 Lumen surface cache 或 hit lighting 得到二次点的 outgoing radiance；
- 未命中时使用天空/远景贡献；
- 结果转换到当前 pre-exposure；
- 对异常大 radiance 做上限约束，默认上限量级为 100，以控制 firefly。

目标函数以 `luminance(OutgoingRadiance)` 为主。这里没有把当前首顶点的 diffuse `NoL/pi` 或粗糙高光响应完全塞进 target；它们在最终 integrate/upsample 阶段按当前材质计算。因此 target 是便宜近似，而不是完整 integrand。

初始 pass 后，reservoir 已是合法的 RIS 表示；如果后续复用全部关闭，它仍可以输出一条随机 GI sample 的估计。

3.4 Temporal Resampling

时间 pass 使用速度/矩阵重投影寻找上一帧 reservoir，围绕落点搜索一个小邻域，源码上限为 3x3/9 个位置。候选历史必须通过表面验证：法线夹角阈值约 25 度，深度阈值约 0.01，并且历史坐标、资源尺寸和 frame state 有效。

历史 radiance 要从上一帧 pre-exposure 变换到当前帧。历史 `M` 受上限约束，默认不超过当前 reservoir `M` 的约 20 倍，避免旧样本无限控制新帧。当前实现通常在找到首个有效历史点后合并，而不是对搜索区的所有

历史 reservoir 都合并。

历史失效条件包括 camera cut、变换重置、buffer 尺寸变化与相关 CVar 配置变化。可选路径会每隔若干帧重新追踪历史 sample，从而刷新动态遮挡或二次命中的 radiance；否则历史只是通过新旧表面一致性继续保留。

有一个值得警惕的源码细节：时间路径预留了 Jacobian 逻辑，但当前某处以常数 1 代替，并带有 firefly/TODO 语义。它说明这条功能仍在迭代，不能把当前代码的每个近似都误认为 ReSTIR GI 理论要求。

3.5 Spatial Resampling

空间 pass 默认执行两轮，每轮从旋转螺旋/黄金角序列选择约 4 个邻居。采样半径按屏幕尺寸缩放，默认比例约 0.05。每个邻居先通过深度和法线一致性检查，再把其路径样本重连到当前首顶点。

重连后计算 ReSTIR GI Jacobian：

```
J = newCos * oldDistanceSquared  
/ (oldCos * newDistanceSquared)
```

miss 样本取 1；极端几何变化会让权重爆炸，因此源码拒绝约 [0.1, 10] 之外的值。这种 rejection 是稳健性近似，也意味着最终结果不再只是教科书公式的机械执行。

为了避免跨薄墙传播，空间 pass 在候选可能胜出时执行一段短的屏幕空间遮挡检查，步数约 8。把检查延迟到“可能替换 reservoir”之后，可以减少不必要查询；它仍不能发现屏幕外遮挡，因此最终质量还依赖 Lumen 后续过程。

两轮空间 pass 使用 ping-pong 资源，保证每轮只读稳定输入。若在单个 UAV 中原地更新，邻居读取顺序会形成不可控反馈，统计意义和平台复现性都会改变。

3.6 Integrate、上采样与材质响应

reservoir 分辨率低于最终输出，因此 UE 不是简单把一个 reservoir 复制到 2x2 像素。Integrate pass 在每个全分辨率像素附近采集多个 reservoir，默认螺旋样本数约 16、kernel 半径约 3，并用接收平面、深度和法线关系计算权重。

每个 reservoir 提供 $\text{OutgoingRadiance} * W$ ；当前全分辨率像素再应用材质响应：

- 漫反射使用近似 NoL / π ；
- 粗糙镜面使用 GGX 分布项与隐式 visibility 近似；
- 同时估计方差，供后端稳定器使用。

这种“先复用 incident/outgoing radiance，再在目标像素应用 BSDF”的分层，对低分辨率 gather 很重要。若缓存的是邻居已乘材质的最终颜色，材质边界会更容易串色。

3.7 第二段时域滤波与双边滤波

reservoir temporal resampling 解决的是采样分布，不等于输出无噪。UE 对最终光照还有第二段历史：重投影 diffuse/specular lighting、验证 disocclusion、做邻域 clamp，历史长度默认上限约 10 帧；随后执行双边滤波，默认邻域约 8。短程 AO 另外补回 GI 大核过滤容易抹掉的接触阴影。

所以整条路径实际上有两种历史：

1. reservoir 历史：决定下一帧采哪个路径样本；
2. lighting 历史：降低最终 RGB 噪声并稳定上采样。

两者的 reset 条件相关但职责不同。把它们合成一张“历史颜色”会失去 reservoir 的统计状态，也无法独立调节采样滞后与显示滞后。

3.8 Lumen 路径对 DSMEngine 的可借鉴点

可直接借鉴的是工程组织：初始、时间、空间、integrate、denoise 分 pass；权重与样本载荷分资源；ping-pong；显式 history validation；pre-exposure；M cap；低分辨率 reservoir 与全分辨率材质重算；调试 CVar。

不能直接借鉴的是样本语义和依赖：DSMEngine 当前没有 Lumen surface cache/hit lighting，目标是直接光，不应保存 GI hit normal/distance 或使用 GI Jacobian，也不需要复现短程 AO 组合。

4. UE 5.8 MegaLights 源码解析

4.1 产品目标与顶层流水线

MegaLights 是面向大量动态直接光的随机光照系统。官方说明和源码都强调固定的每像素光线预算、重要性抽样、ray guiding 以及专用去噪。主要实现分布在：

- **MegaLights.cpp**：开关、资源和顶层调度；
- **MegaLightsSampling.cpp** 与 **MegaLightsSampling.usf/.ush**：灯光候选与 WRS；
- **MegaLightsLightTargetPDF.usf**：目标权重；
- **MegaLightsRayTracing.cpp** 及 RT shader：可见性；
- **MegaLightsResolve.cpp**、**MegaLightsShading.usf/.ush**：分组着色与权重恢复；
- **MegaLightsDenoising.cpp**、**MegaLightsViewState.h**：历史和后端去噪。

顶层近似为：

```
创建临时资源、分类 tiles、重投影历史
-> GenerateSamples
-> VSM page marking (若使用)
-> 一个或多个 RayTrace / Resolve 批次
-> 更新 visible/hidden light hashes
-> Denoise
-> 合成 diffuse/specular direct lighting
```

默认下采样因子为 2、每个工作像素约 4 个 light samples；具体数量、方向光纳入与阴影方法均由 CVar/项目设置控制。方向光默认不纳入 MegaLights，官方也倾向把强主方向光单独处理。

4.2 候选来源：clustered light grid

每个像素从 forward clustered light grid 读取可能影响它的灯光；可选再加入方向光。对每盏灯计算未遮挡目标，包括材质分裂后的 diffuse/specular、距离和形状衰减、lighting channels、light function、IES 等 UE 光照语义。

目标不是简单线性 luminance。源码使用类似 $\log_2(\text{luminance} + 1)$ 的压缩并在很小权重附近平滑截止，以免少数极亮灯垄断所有样本，也减少几乎无贡献灯的随机命中。它更符合感知稳定性，但与无偏的 **pHat=真实标量贡献** 不完全相同。

历史 visible/hidden light hash 进一步调节权重：上一帧长期被挡住的灯降低 proposal 权重，可见灯保留较高机会。这是 UE 所称的 ray guiding。它影响“选谁去追踪”，但最终仍需重新做当前帧可见性。

4.3 向量化 WRS

FLightSampler 同时维护若干选中样本、独立随机数和全局 **WeightSum**。**AddLightSample** 对遍历到的灯做 weighted reservoir sampling:

```
newSum = oldSum + weight
tau = oldSum / newSum
若 random > tau, 则用当前灯替换对应 slot
```

实现可一次维护 1、2 或 4 个独立 slot, 适合 GPU 向量化。遍历全部 tile 灯光后, 选中样本保存 reciprocal selection factor, 概念上是:

```
sampleWeight = totalWeight / selectedWeight
```

面光还保存面内 UV, 样本记录 ray type, 并在后续合并相同灯光/形状样本, 减少重复着色。

一个容易犯的移植错误是: MegaLights 枚举 **N** 盏灯得到精确离散分布 $q_i = \text{weight}_i / \text{sumWeight}$ 后, 又把它硬套成原论文 “**M=N** 个独立源候选” 的 reservoir。对当前像素来说, 这次枚举已经直接生成了一个来自 **q** 的样本; 若要送入后续 ReSTIR reservoir, 应把它视为一个有效 source sample (**M=1**, 源概率为 q_i), 或者完整推导等价 estimator, 不能把 **N** 重复计数。

4.4 样本与光线压缩

生产路径强烈压缩数据。典型 light sample 用 32 位打包灯光索引、状态标志和近似权重; ray record 也可用 32 位打包距离、面内 UV、first-person 标志和 ray type。灯光索引大约占 15 位, 距离使用 half 风格编码, UV 各约 6 位。

这类布局依赖 UE 明确的索引生命周期与容错 clamp。DSMEngine 不应从第一版就照抄位宽: 先用结构化 buffer + float32 验证 PDF、selected ID 和贡献, 再根据捕获的范围直方图设计压缩, 否则很难区分统计错误与量化错误。

4.5 可见性后端

选中样本被转换为 compact rays。阴影后端可以是:

- Virtual Shadow Maps;
- 屏幕空间追踪后接硬件 RT;
- inline/hardware RT 与 Lumen scene 表示;
- 平台允许时的软件追踪表示。

屏幕追踪能快速命中屏幕内已知深度, 但有视野边界与薄几何问题; 硬件 RT 是更可靠的世界空间 fallback。光源本身的 **ShadowMethod**、项目设置与平台能力共同决定路径。这里的可插拔性比某个具体 trace shader 更值得仿照: 采样器只产出 “从表面到灯光的可见性查询”, 不应绑定唯一加速结构实现。

4.6 Resolve 与着色

Resolve 阶段对相同样本分组, 评估完整 UE 直接光语义, 再乘 reciprocal selection weight 并除以样本数。输出拆分 diffuse/specular, 并产生后端需要的 confidence。

源码有多项显式稳健策略: 例如普通 shading weight 上限约 20, 历史判定为 hidden 的样本上限更低 (约 5), 小权重区域做平滑。这些 clamp 有意牺牲严格无偏性, 换取爆点控制、较短历史和可预测的游戏画面。

这与论文偏置变体的共同哲学是“接受可测、可控的偏差换固定预算”，但两者不是同一公式。复刻 MegaLights 时必须把 clamp 作为产品策略参数，而不是偷偷混入 reference path。

4.7 历史到底保存什么

FMegaLightsViewState::FResources 主要保存：

- diffuse/specular lighting history;
- moments 与累计 frame count;
- visible/hidden light hash;
- history depth/normal 等几何验证数据。

它不保存上一帧选中的 light sample、wSum/M/W reservoir。因此下一帧不会把历史样本当作代表旧候选集重新合入；历史只用来指导 proposal 和滤 RGB 输出。这是判断“MegaLights 不是经典时空 ReSTIR”的最直接源码证据。

4.8 去噪

MegaLights 的时间去噪默认可累计约 12 帧，使用深度/法线/距离判定、moments、neighborhood clamp 与 disocclusion 处理。空间阶段使用约 8 像素 kernel、默认若干随机邻居，并对新显露区域增强空间过滤。

去噪器不是 estimator 的替代品：选灯概率和 reciprocal weight 仍要正确，否则去噪只会把偏差变平滑。反过来，即便 estimator 无偏，只有 4 个样本的单帧结果也需要针对 diffuse/specular 的重建。

5. 原论文与 UE 的逐项差异

5.1 与 Lumen ReSTIR Gather

问题	原论文 ReSTIR DI	Lumen ReSTIR Gather	对设计的影响
积分对象	首表面直接光	首表面之后的一次间接路径	不能复用同一 sample payload
候选参数化	光源面积/环境域	首顶点半球方向与二次命中	GI 必须保存 hit geometry
目标函数	未遮挡直接反射	outgoing radiance 亮度近似	最终 BSDF 应用阶段不同
重连	同一光源样本给新接收点	新首顶点连接旧二次点	后者需 Jacobian
依赖	发光几何与可见性	Lumen cache/hit lighting/sky	DSM 无法孤立复制
输出	直接光估计	低分辨率 GI 后重建	去噪与合成位置不同

Lumen 的工程完整度高于论文伪代码，但算法目标已经换了。把它称为“UE 版原始 ReSTIR”是不准确的。

5.2 与 MegaLights

问题	原论文 ReSTIR DI	MegaLights	对设计的影响
初始选择	从 proposal 抽 M 个候选后 RIS	遍历 clustered lights, 向量化 WRS	UE 可得到当前 tile 的精确权重和
时空复用	合并 reservoir	不合并历史 reservoir	UE 的跨帧收益来自 guiding/denoise
历史状态	selected sample + wSum/M/W	lighting/moments/hash/depth/normal	不能用同一 reset/存储契约描述
可见性	选后阴影, 论文讨论传播	多后端追踪, hash 引导	MegaLights 更产品化

问题	原论文 ReSTIR DI	MegaLights	对设计的影响
偏差	分析偏置/无偏两版	感知 target、多种 clamp	UE 更重稳定和固定预算
灯光语义	发光三角形、环境	UE 解析灯、函数、IES、channels	生产灯光系统集成更完整

MegaLights 最值得复用的是产品架构，而不是把名字替换成 ReSTIR：clustered candidate source、向量化 WRS、稳定的 visibility abstraction、split lighting、hash guidance、专用 denoiser和丰富可视化。

6. DSMEngine 当前渲染架构审阅

6.1 现有 frame pipeline

当前 DeferredRenderer 近似按顺序执行：

```
GBuffer
-> MotionVector
-> SSAO
-> Lighting / shadow preparation
-> DeferredLighting
-> Skybox
-> Transparent
-> TAA
-> PostProcess
-> FinalOutput
```

各 pass 通过显式资源状态和 fence 串联，尚无 render graph。ReSTIR 因而需要自己管理多张历史纹理、ping-pong、resize/reset 与 UAV barrier；不能假设 UE RDG 的资源生命周期会自动存在。

6.2 GBuffer 能提供什么

现有 GBuffer 大致为：

- albedo + metallic: **RGBA8**;
- view-space encoded normal: **RG32_FLOAT**;
- roughness + occlusion + emissive: **RGBA8**;
- depth: **D32**。

延迟 shader 已能从 depth 重建 world position/normal，满足解析灯 ReSTIR 的基本 surface input。但当前材质 shader 虽读取 emissive texture，写入 GBuffer 的 emissive 通道仍为 0；因此短期不能把“场景 emissive mesh 自动变成采样灯”作为可用能力。

normal 和 roughness 足以做历史表面验证；若未来支持 thin/translucent 或多 layer GBuffer，需要另设 sample domain，不能把现有 opaque history 硬套过去。

6.3 灯光系统

引擎支持方向光、点光、聚光灯；上限约 4 个方向光、128 个其他灯。DeferredLighting 对方向光顺序循环，对局部灯使用 tile mask，再评估 raster shadow map。

CPU 会按视锥剔除后重新上传局部灯数组，这意味着 GPU index 不具备跨帧稳定性。ReSTIR history 若只保存 index，摄像机稍动就可能把旧 reservoir 解释成另一盏灯。必须增加稳定 **LightId** 和 **Revision**，或建立稳定 ID 到当前 packed index 的 GPU 映射。

另一个实际缺陷是 tile culling shader 中 `gsTileInfo` 的位掩码没有看到可靠的显式清零，只更新 `count/near/far`。把这个 mask 当候选列表前，应先用 GPU capture/测试确认初始化与每 tile 写入协议；否则 ReSTIR 可能读到上一 tile/上一帧残留灯光。

6.4 运动向量与历史

MotionVector 输出 `RG16F`，约定为 `currentUV - previousUV`；shader 使用当前/上一帧矩阵与 mesh buffer，能够表达相机和物体运动。TAA 已有颜色 history，并在 resize 时重置。

ReSTIR 仍缺少独立历史契约：camera cut 标志、上一帧 depth/normal、stable light mapping、场景 revision、reservoir size 与 downsample jitter。不能只复用 TAA 的“有一张 history color”布尔状态。时间重投影应明确写成：

```
previousUV = currentUV - motionVector
```

并用可视化测试移动摄像机和移动物体，确认符号、jitter 去除与坐标原点一致。

6.5 DXR 基础设施

底层 Graphics 层已有 BLAS/TLAS 抽象、ray tracing pipeline、shader binding table 与 `DispatchRays`；项目中的独立 RayTracing sample 曾完成构建验证。D3D12 feature query 也能检查 RayQuery tier 1.1。

缺口在 renderer 集成：DeferredRenderer 没有场景级 BLAS 缓存、instance transform/update、每帧 TLAS build/refit、材质 alpha policy 和阴影 query 接口。底层“能 DispatchRays”不等于当前场景“已有可追踪 TLAS”。此外尚未发现被主渲染器验证过的 inline `RayQuery` 路径，所以它应作为技术 spike，而不是计划中的既定事实。

现有静态 mesh 顶点 position 位于 buffer 可识别 offset，index 支持 16/32 位，这让按 mesh/submesh 构建 BLAS 可行。第一阶段只纳入静态、不透明、双面策略明确的实例；skinned、alpha mask、transparent 以后再扩展。

6.6 TAA 与 ReSTIR 去噪的相互作用

若 ReSTIR 自己已有 lighting temporal filter，随后全画面 TAA 再累计，会出现双重历史：拖影、过度 clamp 或响应迟钝。实现时应把 direct-light confidence/variance 纳入 TAA 调参，至少提供关闭 ReSTIR lighting history、关闭 TAA accumulation 的 A/B 模式，分别定位 reservoir history、lighting denoiser 和最终 TAA 的问题。

7. 建议先冻结的外部契约

按照“先对齐边界，再替换内部实现”的原则，建议目标定义为：

7.1 第一版功能范围

- 只处理 deferred opaque surface；
- 支持 point/spot local lights；
- directional lights 保持现有 raster shadow path；
- 不支持透明、alpha mask、area light、emissive triangle、GI；
- 输出与现有延迟光照可比较的 diffuse/specular direct lighting；
- 每个最终选中 local light 做一条 ray-traced visibility query；

- 无 DXR 或功能关闭时，完全回退现有 DeferredLighting;
- reference mode 可对所有影响灯逐一追踪，作为离线/小场景真值。

这里暂不处理 area/emissive 不是因为 ReSTIR 不适合，而是当前引擎没有相应灯光采样域和稳定资源语义。先把离散解析灯 estimator 做对，未来再增加形状条件 PDF。

7.2 质量契约

- 静态场景长时间平均应接近 brute-force reference;
- camera cut/resize 后首帧不得读取旧 reservoir;
- 灯光添加、删除、排序、出入视锥不能错误引用另一盏灯;
- 移动物体与 disocclusion 在有限帧内恢复，不出现永久漏光;
- 任何 clamp、history visibility guidance、PDF 近似都必须可开关并标为 bias mode;
- debug view 必须显示 selected light、source PDF、pHat、wSum/M/W、history validity、visibility 与 rejection reason。

7.3 性能契约

性能目标不应写成“必然比 tiled lighting 快”。建议记录：

- initial/temporal/spatial/trace/shade/denoise 各 pass GPU 时间;
- 每输出像素候选数、有效历史率、实际 rays/pixel;
- reservoir 与历史显存;
- 16、64、128 灯以及未来更高灯数的缩放曲线;
- raster shadow map 路径、全灯 ray reference、ReSTIR 三者的同质量对比。

8. 推荐的 DSMEngine 架构

8.1 模块分层

建议把职责分成四层：

```
Scene RT Layer
  BLAS cache / instances / TLAS / visibility query
  |
Light Sampling Layer
  stable light table / tile candidates / source PDF / pHat
  |
ReSTIR DI Layer
  initial -> temporal -> spatial -> visibility -> shade
  |
Reconstruction Layer
  temporal lighting -> variance/confidence -> spatial filter -> TAA
```

分析层面的候选文件位置为：

- DSMEngine/Runtime/Render/RayTracing/RayTracingScene.*：renderer 级 AS 生命周期;
- DSMEngine/Runtime/Render/DeferredRenderer/ReSTIRDirectLightingPass.*：pass 编排和历史;

- **DSMEngine/Shaders/DeferredShader/ReSTIR/**: initial、temporal、spatial、visibility/shade、denoise;
- 现有 light upload/culling: 增加稳定 ID、revision 与候选 PDF 需要的数据;
- 现有 motion/TAA: 增加明确 camera cut 与历史协调接口。

这些只是建议边界，不代表现在应创建文件。真正实施前应写独立 ExecPlan 并先确认最终输出如何替换现有 local-light contribution。

8.2 Reservoir 参考布局

第一版应优先可验证：

```
ReservoirSample:
uint stableLightId
uint lightRevision
float2 shapeUV // 点/聚光灯暂未使用，保留语义
uint flags

ReservoirStats:
float wSum
float M
float W
float selectedPHat // 可选调试；复用时仍按当前像素重算
```

不要缓存邻居的最终 RGB 作为 sample truth。每次复用和最终着色都从当前 surface、当前 light table 重建未遮挡贡献。M 初期使用 float/uint32，避免压缩饱和和悄悄改变统计。

8.3 稳定灯光表

CPU scene light 应拥有跨帧不变的 32 位 ID；光强、位置、方向、范围、形状或采样参数的语义变化递增 revision。GPU 每帧上传：

- dense current light data;
- stable ID -> dense index 映射，或 dense entry 内携带 stable ID 并用 GPU hash 查找;
- active/revision 状态;
- tile candidate mask/list。

仅灯光移动是否使历史 sample 失效取决于 sample 语义。点光只保存 stable ID，可以在当前位置重算 target，通常无需失效；面光若保存旧表面 UV，形状拓扑改变则必须失效。把这一区别写进 revision policy，避免过度 reset 或错误复用。

9. 建议算法：从 reference 到 UE 风格

9.1 Pass 0: brute-force reference

先做一个只在调试模式运行的 reference：遍历当前 pixel/tile 所有 local lights，对每盏灯评估完整 unshadowed RGB，并追踪可见性，求和。灯数低时它很慢但概念简单，是验证后面所有概率权重的依据。

需要两个 reference 版本：

1. unshadowed reference: 排除 TLAS/自相交问题，只验证选灯 estimator;
2. visible reference: 每灯阴影 query，验证最终成像。

如果 ReSTIR 与 unshadowed reference 已不一致，就不要先调 denoiser 或 RT bias。

9.2 Pass 1: Initial RIS

每像素从 tile candidate list 产生 **M0** 个候选。最简单的 source distribution 是在有效局部灯中均匀选择：

```
pSource(light_i) = 1 / lightCount
pHat_i = luminance(max(unshadowedDiffuse_i + unshadowedSpecular_i, 0))
w_i = pHat_i / pSource_i
```

使用独立或低差异随机数流式更新 reservoir，最后算 **W**。target 不含 visibility，因而 initial pass 无需追踪 **M0** 条阴影光线。若 tile 无灯或全部 **pHat=0**，写空 reservoir。

均匀 source 容易验证但对功率差异大的灯方差高。第二步才改为按灯光功率/范围的 alias table，或 MegaLights 式枚举全部 tile lights 后 WRS。每次改 proposal 都要输出并验证真实 source probability。

9.3 枚举式 WRS 的正确接法

若 tile 最多 128 灯，直接遍历并算 target 的成本可能可接受。遍历产生精确 **sumPHat**，选中灯分布为：

```
q(selected=i) = pHat_i / sum_j(pHat_j)
```

这时单样本估计 **f_i / q_i** 已合法。若把它作为 temporal/spatial ReSTIR 的当前来源，可构造代表一个 source draw 的 reservoir：

```
M = 1
wSum = pHat_i / q_i = sumPHat
W = wSum / (M * pHat_i) = 1 / q_i
```

不要令 **M=lightCount**，除非使用的确实是原论文那套“每盏灯作为一个确定候选集合”并完成对应 generalized RIS/MIS 推导。第一版随机取 **M0** 个独立候选更容易与论文公式——对照。

9.4 Pass 2: Temporal Resampling

读取 motion vector 得到历史 UV，加载 reservoir 和 history depth/normal。验证建议：

- UV 在历史 viewport 内；
- relative depth error 小于可调阈值；
- normal dot 大于可调阈值；
- stable light ID 在当前表存在且 revision 合格；
- 当前与历史 reservoir resolution、jitter phase、scene epoch 一致。

合并伪代码：

```
R = currentReservoir
H = validatedHistoryReservoir

H.M = min(H.M, currentReservoir.M * 20)
pHatH = EvaluateTarget(currentSurface, Resolve(H.lightId))
wH = pHatH * H.W * H.M
UpdateReservoir(R, H.sample, wH, H.M, random)
FinalizeW(R, EvaluateTarget(currentSurface, R.sample))
```

点/聚光灯直接光不计算 Lumen GI Jacobian。光源移动后按当前 light data 重算 target；遮挡移动不靠历史 visibility 判真，最终仍追踪。

9.5 Pass 3: Spatial Resampling

第一版只做一轮、4 个邻居、较小半径，使用 blue-noise/黄金角旋转。验证 depth/normal，并避免读取自己正在写的 UAV；源 reservoir 与目标 reservoir 分离。

每个邻居的 merge 权重仍为：

```
wNeighbor = pHat_current(neighbor.sample)
* neighbor.W * neighbor.M
```

不需要 GI Jacobian。为了控制重复候选相关性，先限制为一轮；随后通过静态 reference 的多帧均值判断第二轮是否产生不可接受的能量偏差。几何边缘可以引入 plane distance，而不是只看线性 depth。

9.6 Pass 4: Visibility 与 Shade

对最终 reservoir 解析 stable light，重算完整 RGB direct contribution。阴影 query：

```
origin = worldPosition + geometricNormal * normalBias
direction = normalize(lightPosition - origin)
TMin = configurableEpsilon
TMax = lightDistance - endpointEpsilon
```

点/聚光灯只需 boolean occlusion payload。若 inline RayQuery spike 在目标 GPU、驱动与 shader compiler 上通过，compute pass 可以直接 query TLAS；否则使用最小 DXR raygen/miss/any-hit 管线输出 visibility。第一版只收录 opaque geometry，可使用 accept-first-hit/end-search；alpha mask 以后才需要 any-hit 材质判断。

最终结果：

```
directRGB = visible ? FullUnshadowedRGB(currentSurface, light) * R.W : 0
```

若 pHat 是 luminance 而 RGB 是向量，R.W 已包含 target 归一化，不应再除一次 luminance。用白灯/红灯/蓝灯分别与 reference 对比可以很快发现通道或重复除法错误。

9.7 Pass 5: 重建与去噪

先在全分辨率实现 reservoir，排除上采样变量。正确后再尝试 half-resolution：

- reservoir history 保持低分辨率；
- full-res pixel 在附近收集多个 reservoir；
- 按 depth、normal、plane distance 和材质粗糙度加权；
- 在 full-res 当前表面重算 BSDF，不复制邻居最终颜色；
- 输出 diffuse/specular、moments、variance、confidence。

lighting temporal filter 使用独立 history depth/normal 与 neighborhood clamp，最大历史帧数先设较短；disocclusion 增强空间过滤。与 TAA 联调时保留四种模式：均关闭、只 reservoir history、加 lighting history、再加 TAA。

9.8 Pass 6: 吸收 MegaLights 优化

在 reference 可验证后，按收益逐项加入：

1. tile 全枚举 + 向量化多 slot WRS;
2. visible/hidden light hash 只用于 proposal guidance;
3. compact sample/ray;
4. 相同灯光 sample 合并;
5. VSM 与 RT visibility 可切换;
6. 感知 target、shading weight clamp、hidden clamp;
7. 更成熟的 split diffuse/specular denoiser。

每项都必须有前后误差和 GPU 时间。特别是 128 灯场景，枚举 target 的 ALU/带宽可能超过随机候选的收益；不能因为 UE 使用就默认适合 DSMEngine。

10. 分阶段实施路线（分析，不执行）

阶段 A: 契约与诊断基础

交付目标：没有 ReSTIR 图像，但所有输入可验证。

- 定义 local direct lighting 替换/合成边界，避免现有 DeferredLighting 再算一次同样贡献；
- 增加 stable light ID/revision;
- 修正并验证 tile light mask 清零和边界；
- 建立 camera cut、resize、scene epoch 历史 reset;
- 增加 selected light/target/PDF debug views;
- 保存 brute-force reference 捕获。

完成标准：灯光重排、相机运动与 resize 时，GPU stable ID 可视化始终指向同一语义灯光。

阶段 B: Renderer 级 ray tracing scene

交付目标：任意 deferred opaque pixel 能可靠查询到一条 local-light visibility。

- 静态 mesh BLAS cache;
- instance transform 与 TLAS build/refit;
- 每帧资源 lifetime、barrier、fence;
- normal bias、TMin/TMax 与双面策略;
- inline RayQuery spike 和 DXR DispatchRays fallback 比较;
- RT 开关与无支持硬件 fallback。

完成标准：用简单遮挡场景与 raster shadow reference 比较，无明显自阴影、穿透或上一帧 AS 延迟。

阶段 C: 单帧正确性

交付目标：Initial RIS + 一条可见性光线，无时间/空间历史。

- 均匀 source, 明确 PMF;
- float32 reservoir;
- full-resolution;
- unshadowed/visible brute-force reference;
- 固定随机种子和多帧平均误差工具。

完成标准：静态场景平均值与 reference 一致；0 灯、1 灯、不同颜色和极端强度不出 NaN/Inf。

阶段 D：时间复用

交付目标：历史 reservoir 合并与严格 reset。

- history depth/normal/reservoir;
- motion reproject;
- stable light resolve;
- **M** cap;
- disocclusion 与 camera cut;
- history validity debug view。

完成标准：静态噪声下降；移动相机/物体/灯后无永久错误，reset 首帧等价于 initial-only。

阶段 E：空间复用

交付目标：一轮邻域合并，边缘可控。

- ping-pong;
- 低差异邻居;
- depth/normal/plane rejection;
- pass/radius/sample count 可调;
- bias 与 reference 的长期平均对比。

完成标准：平面区域方差下降，薄墙和轮廓无明显跨面漏光；偏差量有记录。

阶段 F：重建与 UE 风格优化

交付目标：产品级成本/画面折中。

- half-resolution 与材质感知 upsample;
- diffuse/specular lighting history;
- moments/confidence;
- MegaLights WRS/history guiding/压缩按实测引入;
- TAA 协调;
- 性能和显存预算。

完成标准：在目标硬件/场景上，同等质量优于选定基线，并且各偏置开关的影响可解释。

阶段 G：扩展采样域

只在前述契约稳定后考虑：

- sphere/rect/disk area light：加入形状 UV 和条件面积 PDF；
- emissive mesh：建立 emissive primitive table、功率分布、alias table 与动态更新策略；
- environment proposal；
- alpha-mask RT visibility；
- skinned/dynamic BLAS；
- 多 reservoir/多 selected samples；
- 无偏 MIS 修正或更先进的 generalized resampling。

GI 不属于这个阶段的自然小扩展。它需要路径样本、二次点着色、Jacobian、世界空间可见性和完全不同的降噪问题，应单独立项。

11. 验证矩阵

11.1 正确性场景

场景	主要验证点	失败特征
0/1 盏灯	空 reservoir、权重基线	NaN、非零鬼光
128 等强灯	均匀 PMF、selected histogram	某些 ID 永不被选
1 盏极亮 + 多暗灯	重尾、clamp、wSum	firefly 或系统丢能量
红/绿/蓝灯	luminance target 与 RGB 恢复	色偏、重复除 target
灯光添加/删除/重排	stable ID/revision	历史突然变成别的灯
移动灯和遮挡物	当前 target/visibility 重算	阴影滞留、历史假可见
薄墙两侧强灯	spatial rejection	跨墙漏光
相机 cut/resize	history epoch	满屏旧 reservoir
移动物体/disocclusion	motion 与 history validation	拖尾、黑洞
光滑/粗糙材质	specular target 与滤波	高光丢失或扩散

11.2 数值指标

- 对静态 reference 计算多帧平均 RGB relative error、MSE/PSNR；
- 分别报告 bias（平均差）与 variance，不能只报最终 TAA 后 PSNR；
- 统计 M、W、wSum、source PDF、selected light frequency 的直方图；
- 统计 history accept/reject 原因和 disocclusion 比例；
- 检测 GPU buffer 中 NaN/Inf 与过大权重；
- 固定随机种子保存回归截图，同时另跑随机序列避免只对一个 seed 优化。

11.3 性能指标

- GPU timestamp 分 pass；

- TLAS build/refit 与 trace 分开;
- rays/pixel 和 any-hit invocation;
- reservoir/history/denoiser 显存;
- 候选数和邻居数扫描曲线;
- 1080p、1440p、4K 与 full/half resolution;
- 至少两档 RT 能力 GPU;
- 相同画质下对比 raster shadow、全灯 RT reference、initial-only、temporal、temporal+spatial。

12. 主要风险与应对

风险 1：在 128 灯上没有性能收益

现有 tiled raster loop 可能更便宜，尤其局部灯没有昂贵阴影时。应把首个价值目标定为 ray-traced local shadows 与固定光线预算，并保留 raster fallback；用真实 target scene 决定是否默认开启。

风险 2：TLAS 成本吞掉采样收益

场景级 AS 是当前最大基础缺口。先测静态 BLAS + TLAS refit/build 成本，再决定动态物体支持范围。不能只计 trace pass 而忽略 AS update。

风险 3：历史灯光索引失配

这是最容易造成“画面偶尔爆亮却难复现”的问题。stable ID/revision 必须先于 reservoir history 实现，并有可视化和添加/删除灯回归。

风险 4：重复累计和不可解释偏差

spatial 多 pass、MegaLights clamp、history guiding、lighting denoise 与 TAA 都会改变结果。逐项开关、reference path 和阶段化引入是唯一可靠的定位方法。

风险 5：高光重尾与低分辨率漏样

镜面 target 对视角和法线极敏感，邻居 reservoir 可能对当前点重要性突变。第一版 full-res；后续 upsample 按 roughness 缩小核，diffuse/specular 分离，必要时为 specular 使用更多 slot 或不同 target。

风险 6：误用 GI Jacobian

DSM 第一阶段是解析灯 DI。应在 shader 注释和设计文档明确“直接光灯域复用不使用 Lumen GI Jacobian”，防止照抄 UE 函数造成距离/余弦重复校正。

风险 7：RayQuery 平台路径未验证

feature tier 查询只说明 API 支持，不说明引擎 shader 编译、descriptor binding、TLAS lifetime 已打通。保留短期 spike 和 DXR raygen fallback，先用最小 boolean payload 验证。

13. 参数起点与调优顺序

以下只建议作为第一轮实验点，不是最终默认值：

参数	正确性起点	产品探索	理由
Reservoir 分辨率	full	half	先排除上采样影响

参数	正确性起点	产品探索	理由
Initial candidates	8	4-32 扫描	比论文 32 更适合当前 128 灯边界做起点
Temporal history	off -> on	M cap 20x	先建立单帧 reference
Spatial passes	0 -> 1	1-2	多轮更易产生相关偏差
Neighbors/pass	4	4-8	与 UE/Lumen 的工程量级接近
Normal threshold	dot 25°	按 roughness 调节	防跨面复用
Depth threshold	relative 1-10% 扫描	plane distance	投影尺度相关
Visibility rays	1/selected sample	1-4 slots	固定预算核心
Lighting history	off	4-12 frames	避免初期双历史
Weight clamp	off	基于直方图	reference 必须无隐藏 clamp

调优顺序应是：source PDF 正确性 -> target 与 reference -> temporal validation -> spatial rejection -> RT bias -> denoise -> TAA -> 压缩/clamp。反过来先调 denoiser，常会把概率错误暂时藏起来。

14. 是否应“完全仿照 UE”

结论是不应逐行仿照，但应分层借鉴。

应仿照

- Lumen 的 pass 分解、历史资源与 ping-pong;
- reservoir history 和 lighting history 分离;
- 低分辨率 sample 在 full-res 当前材质重算;
- camera cut/resize/pre-exposure/M cap 等稳定性契约;
- MegaLights 的 clustered candidate source、向量化 WRS 和 visibility abstraction;
- diffuse/specular 分离、moments/confidence 和调试视图;
- 把 “reference unbiased-ish path” 与 “产品 biased path” 分别维护。

不应仿照

- 不复制 Lumen surface cache/hit-lighting 依赖;
- 不把 ReSTIR GI Jacobian 用到解析灯 DI;
- 不直接复制 UE 的 32 位 packing 和 clamp 数值;
- 不假设 UE 的下采样、历史长度适合 DSM 的分辨率和 TAA;
- 不把 MegaLights visible/hidden hash 当成真实可见性缓存;
- 不把本地 UE 5.8.1 分支的 TODO/定制代码当成稳定公开契约。

推荐的组合可以概括为：

原论文提供 estimator 数学
+
Lumen 提供 reservoir pipeline 组织
+
MegaLights 提供产品化直接光工程经验
+
DSMEngine 自己的 light / DXR / TAA 契约

15. 决策门

真正进入实现前，建议团队明确回答：

1. 第一版主要目标是“更多灯”还是“固定预算 RT 阴影”？当前上限 128 时，后者更有说服力。
2. local light 的现有 raster shadow 是否被替换、混合还是只作为 fallback？不能让两条路径重复贡献。
3. 目标硬件是否强制 DXR 1.1？若否，fallback 画质和功能边界是什么？
4. 是否接受 bias mode 为产品默认？若接受，允许的长期能量误差如何量化？
5. first release 是否真的需要 half-resolution？若工期有限，应优先 full-res correctness。
6. stable light ID 由 scene 层还是 render proxy 层拥有？生命周期必须跨 CPU culling 稳定。
7. 动态 mesh/alpha mask 的 RT 支持是否是首版硬需求？它会显著扩大 BLAS 与 any-hit 范围。

如果这些问题没有答案，reservoir shader 写得再完整也会在集成阶段反复推倒。

16. 最终建议

对 DSMEngine 的最短可靠路线是：

1. 先把 stable light identity、tile list 正确性和 renderer TLAS 做成独立、可测试能力；
2. 用 full-resolution、float32、均匀 source 实现 initial-only ReSTIR DI reference；
3. 用 brute-force unshadowed/visible 两套真值证明公式和 RT 分别正确；
4. 再加入 temporal 和单轮 spatial reservoir merge，明确保持 DI light-domain 数学；
5. 最后吸收 MegaLights 的多 slot WRS、history guiding、压缩和去噪，并把每个近似作为可开关 bias feature；
6. 只有性能曲线证明收益后，才替换默认 local-light 路径。

若只追求近期画面收益，也可以先做“MegaLights-lite”：tile 灯全枚举 WRS、每像素固定 1-4 条 RT 阴影、lighting denoise，不做历史 reservoir。这比完整 ReSTIR 更短，却不会获得真正的跨帧候选集复用。两者应作为清楚的产品选择，而不是用同一个名称混合实现。

17. 取证来源与版本说明

论文与官方资料

- Benedikt Bitterli 等，Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting, ACM TOG 39(4), 2020, DOI [10.1145/3386569.3392481](https://doi.org/10.1145/3386569.3392481)。
- Y. Ouyang 等，ReSTIR GI: Path Resampling for Real-Time Path Tracing, 2021；本文只用它核对 GI Jacobian 与路径样本差异。

- Epic Games 官方文档：MegaLights in Unreal Engine；用于核对产品定位、阴影后端、ray guiding 与平台说明。
- Epic Games 官方文档：Lumen Global Illumination and Reflections；用于核对 Lumen 产品边界。

UE 5.8.1 本机源码

- Engine/Source/Runtime/Renderer/Private/Lumen/LumenReSTIRGather.cpp/.h
- Engine/Shaders/Private/Lumen/LumenReSTIRGather.usf
- Engine/Source/Runtime/Renderer/Private/Lumen/LumenScreenProbeGather.cpp
- Engine/Source/Runtime/Renderer/Private/Lumen/LumenViewState.h
- Engine/Source/Runtime/Renderer/Private/MegaLights/*.cpp/.h
- Engine/Shaders/Private/MegaLights*

本机路径为 G:\Works\QSClnt。Build.version 显示 5.8.1、兼容 CL 55116800，但文件中存在 QQSpeed 定制注释；因此本文对算法结构的判断来自实际源码，对未来 UE 小版本默认参数不作保证。

DSMEngine 源码范围

- DeferredRenderer 的 frame pipeline、GBuffer、Lighting 与 TAA pass；
- local/directional light 数据和 tile culling shader；
- MotionVector shader 与历史管理；
- Graphics/D3D12 ray tracing abstraction；
- StaticMesh buffer/index layout 与现有 RayTracing sample 记录。

本次只读分析，没有修改 DSMEngine 源码，也没有声称主渲染器的 RayQuery/TLAS 已经可直接使用。所有建议都应在后续实现任务的 ExecPlan 中重新以当前源码状态核验。